Introduction to NumPy Arrays

What is NumPy?

NumPy stands for **Numerical Python**. It is one of the most popular and powerful Python libraries used for numerical computing and scientific calculations.

While Python lists are flexible and easy to use, they become slow and memory-intensive when working with large datasets. NumPy solves this problem by introducing **arrays**, which are much faster, consume less memory, and provide powerful mathematical operations.

NumPy is the foundation for many other Data Science and Machine Learning libraries such as:

- Pandas
- Matplotlib
- Scikit-learn
- TensorFlow
- PyTorch

Think of NumPy as the mathematical engine of Python.

Why Do We Need NumPy?

Suppose you have marks of 10 lakh students.

Using Python lists:

- More memory is used
- Calculations are slower
- Mathematical operations require loops

Using NumPy arrays:

- Less memory
- Faster execution
- Vectorized mathematical operations
- Built-in scientific functions

Features of NumPy

- Fast execution
- Memory efficient
- Multi-dimensional arrays
- Mathematical functions
- Statistical functions
- Linear algebra support
- Random number generation
- Broadcasting
- Array indexing and slicing

Installing NumPy

If NumPy is not installed, use:

pip install numpy

Check installation:

import numpy as np

print(np.__version__)

Example Output

2.x.x

Importing NumPy

The standard way is:

import numpy as np

Why use **np**?

Instead of writing

numpy.array()

we simply write

np.array()

This is the standard convention followed worldwide.

What is an Array?

An array is a collection of elements stored together in contiguous memory.

Unlike Python lists:

- All elements generally have the same data type.
- Faster processing.
- Better memory utilization.

Example

Python List

[10, 20, 30, 40]

NumPy Array

[10 20 30 40]

Creating Your First NumPy Array

```
import numpy as np

arr = np.array([10,20,30,40])
```

print(arr)

Output

[10 20 30 40]

Type of NumPy Array

```
import numpy as np


arr = np.array([1,2,3])


print(type(arr))
```

Output

<class 'numpy.ndarray'>

Notice that NumPy arrays belong to the class:

numpy.ndarray

Creating Arrays from Lists

```
import numpy as np


numbers = [5,10,15,20]


arr = np.array(numbers)


print(arr)
```

Output

[5 10 15 20]

Creating Arrays from Tuples

```
import numpy as np

data = (2,4,6,8)

arr = np.array(data)

print(arr)
```

Output

```
[2 4 6 8]
```

One-Dimensional Array (1D Array)

A 1D array contains only one row.

Example

```
import numpy as np

arr = np.array([10,20,30,40])

print(arr)
```

Visualization

10 20 30 40

Two-Dimensional Array (2D Array)

A 2D array contains rows and columns.

Example

```
import numpy as np

arr = np.array([
  [1,2,3],
  [4,5,6]
])

print(arr)
```

Output

```
[[1 2 3]
[4 5 6]]
```

Visualization

1	2	3
4	5	6

Three-Dimensional Array (3D Array)

A 3D array contains multiple matrices.

Example

```
import numpy as np

arr = np.array([
  [
```

```
    [1,2],
    [3,4]
  ],
  [
    [5,6],
    [7,8]
  ]
])

print(arr)
```

Output

```
[[[1 2]
 [3 4]]

[[5 6]
 [7 8]]]
```

Number of Dimensions

Use

ndim

Example

```
import numpy as np

a = np.array([1,2,3])

print(a.ndim)
```

Output

1

Example

```
b = np.array([[1,2],[3,4]])


print(b.ndim)
```

Output

2

Example

```
c = np.array([
  [[1,2],[3,4]]
])


print(c.ndim)
```

Output

3

Shape of an Array

The shape tells us how many rows and columns an array contains.

Example

```
import numpy as np


arr = np.array([
  [1,2,3],
  [4,5,6]
])
```

```
print(arr.shape)
```

Output

(2,3)

Meaning:

- 2 rows
- 3 columns

Size of an Array

Size gives the total number of elements.

Example

```
import numpy as np


arr = np.array([
  [1,2,3],
  [4,5,6]
])


print(arr.size)
```

Output

6

Data Type of Array

Use

dtype

Example

```
import numpy as np

arr = np.array([10,20,30])

print(arr.dtype)
```

Output

```
int64
```

Example

```
arr = np.array([1.5,2.6,3.8])

print(arr.dtype)
```

Output

```
float64
```

Creating Arrays with Specific Data Types

Example

```
import numpy as np

arr = np.array([1,2,3], dtype=float)

print(arr)
print(arr.dtype)
```

Output

```
[1. 2. 3.]
float64
```

Memory Efficiency

Python List

10

20

30

40

Every element stores:

- Value
- Data type
- Reference

NumPy Array

10 20 30 40

Stores only values in contiguous memory.

Result:

- Faster processing
- Less memory
- Better CPU cache performance

Advantages of NumPy Arrays over Python Lists

Python List	NumPy Array
Slower	Faster
More memory	Less memory
Cannot directly add two lists element-wise	Supports vectorized operations
Loops required	No loops needed for most operations

Limited mathematical operations	Hundreds of built-in mathematical functions

---

Comparison Example

Python List

```
a = [1,2,3]


b = [4,5,6]


result = []


for i in range(len(a)):
  result.append(a[i]+b[i])


print(result)
```

Output

[5,7,9]

NumPy

```
import numpy as np


a = np.array([1,2,3])


b = np.array([4,5,6])


print(a+b)
```

Output

[5 7 9]

Notice how no loop is required.

Real-Life Applications of NumPy

NumPy is used in:

- Data Science
- Artificial Intelligence
- Machine Learning
- Deep Learning
- Image Processing
- Computer Vision
- Robotics
- Finance
- Scientific Research
- Weather Forecasting
- Medical Data Analysis
- Big Data Analytics

Best Practices

- Always import NumPy as np.
- Use arrays instead of lists for numerical computations.
- Prefer vectorized operations over loops for better performance.
- Keep all elements of an array in the same data type whenever possible.
- Use built-in NumPy functions instead of writing manual calculations.

Summary

In this chapter, you learned:

- What NumPy is
- Why NumPy is used
- Installing and importing NumPy

- Creating NumPy arrays
- 1D, 2D, and 3D arrays
- Array dimensions (`ndim`)
- Array shape (`shape`)
- Array size (`size`)
- Array data types (`dtype`)
- Creating arrays with custom data types
- Advantages of NumPy over Python lists
- Real-world applications of NumPy

Key Takeaways

- **NumPy** is the core library for numerical computing in Python.
- **Arrays** are faster and more memory-efficient than Python lists.
- NumPy supports **vectorized operations**, eliminating the need for many explicit loops.
- Understanding array creation, dimensions, shape, size, and data types is the first step toward mastering data analysis and machine learning with Python.
- NumPy serves as the foundation for advanced libraries like **Pandas**, **Matplotlib**, **Scikit-learn**, and **TensorFlow**, making it an essential skill for every aspiring Data Scientist and AI Engineer.